# Graph-Weighted Canonical Correlation Analysis: A Closed-Form Method for Inductive Attributed Graph Embedding

*Proceedings Track Submission*

**Anonymous Author(s)**
Anonymous Affiliation
Anonymous Email

## Abstract

We introduce **Graph-Weighted Canonical Correlation Analysis (GW-CCA)**, a closed-form method for inductive attributed graph embedding. GW-CCA modifies classical CCA by inserting a random-walk graph similarity matrix into the cross-covariance, so that cross-view alignment is weighted by graph proximity rather than restricted to the same node across views. For attributed graphs with a single feature matrix, we construct the second view by multi-hop graph propagation of the features, yielding a linear, non-iterative method that requires only a feature-dimensional eigendecomposition rather than GNN training. Across five inductive benchmarks, GW-CCA is competitive with or stronger than representative GNN and graph self-supervised baselines for node classification, and achieves the best link-prediction performance in our evaluation while requiring orders of magnitude less training time. The method recovers standard CCA when graph weighting is removed and provides a simple alternative to deep graph self-supervised learning for large-scale inductive representation learning.

## 1 Introduction

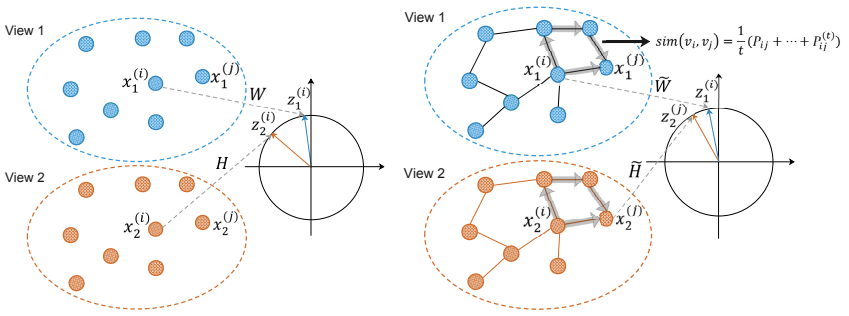
Graph representation learning maps nodes to low-dimensional vectors that encode both graph structure and node attributes, enabling downstream tasks such as node classification and link prediction. In the *inductive* setting, a model trained on one graph, or on a training subgraph, must generate representations for previously unseen nodes. Classical shallow methods such as DeepWalk [1], node2vec [2], LINE [3], and their matrix-factorisation interpretations [4] are efficient, but they learn per-node embeddings and are therefore transductive unless retrained or equipped with an additional out-of-sample extension.

Recent progress in inductive graph representation learning has instead been driven largely by deep self-supervised graph neural networks, including contrastive [5, 6], mutual-information [7, 8], CCA-inspired [9, 10], and masked-autoencoding methods [11]. Although effective, these methods require stochastic optimisation, augmentation design, learning-rate schedules, epoch tuning, and substantial GPU training time. Linear propagation methods such as SGC [12], and its extensions SIGN [13] and $S^2$GC [14], avoid deep message passing and are inductive by construction — but they are typically used as supervised task-specific encoders, not as closed-form unsupervised multiview embedding methods. This raises a natural question: can inductive attributed graph embeddings be learned with a simpler closed-form method while still exploiting graph structure?

We answer this question with **Graph-Weighted Canonical Correlation Analysis (GW-CCA)**[1], a closed-form method for inductive attributed graph embedding. Standard CCA learns linear projections that align paired points $x_1^{(i)}$ and $x_2^{(i)}$ from two views by maximising $\mathrm{Tr}(W^\top X_1^\top X_2 H)$. GW-CCA replaces the ordinary cross-covariance $X_1^\top X_2$ with a graph-weighted cross-covariance $X_1^\top M X_2$,

[1] Code and reproducibility artifacts: https://anonymous.4open.science/r/gw-cca-6151.

Submitted to the Fifth Learning on Graphs Conference (LoG 2026, Proceedings Track). Do not distribute.




**(a)** Standard CCA **(b)** GW-CCA

**Figure 1:** Standard CCA vs. GW-CCA. (a) Standard CCA aligns same-index pairs across two views after linear projection. (b) GW-CCA aligns cross-view node pairs according to graph similarity: larger $M_{ij} = \mathrm{sim}(v_i, v_j)$ gives greater weight to the dot-product alignment $z_1^{(i)\top} z_2^{(j)}$.

where $M_{ij}$ measures random-walk similarity between nodes $v_i$ and $v_j$ (e.g., $M = (P + P^2)/2$ with $P = D^{-1}A$) (Figure 1). Node $i$'s representation in one view is thus aligned with node $j$'s representation in the other view in proportion to their graph proximity, rather than only same-index pairs. For attributed graphs with a single feature matrix $X$, we construct the second view deterministically as $X_2 = \widehat{P}^K X$, where $\widehat{P} = (D+I_n)^{-1}(A+I_n)$ is the self-looped, row-normalised adjacency (an SGC-inspired propagation kernel, a row-stochastic self-loop variant of the symmetric kernel of Wu et al. [12]). GW-CCA then solves a single feature-dimensional ($p \times p$) decomposition to obtain the projections; the method is non-iterative, deterministic, and inductive by construction, embedding unseen nodes by propagating their features on the observed graph and applying the learned projections. In particular, GW-CCA is a strict generalisation of standard CCA [15], recovered when $M = I_n$.

Across five inductive benchmarks — ogbn-arxiv, Reddit, Yelp, ogbn-products, and PPI — GW-CCA is competitive with or stronger than representative GNN and graph self-supervised baselines for node classification, and achieves the best link-prediction performance in our evaluation, while requiring orders of magnitude less encoder-training time on a single CPU (Table 5). These results suggest that graph-weighted closed-form CCA can provide a simple and effective alternative to deep graph self-supervised learning for large-scale inductive attributed graph embedding.

**Contributions.**

- A *graph-weighted cross-covariance* objective (§3) that turns CCA into an inductive attributed graph embedding method, with standard CCA recovered when $M = I_n$.
- A closed-form solver based on a single feature-dimensional ($p \times p$) decomposition, together with a deterministic two-view construction $X_2 = \widehat{P}^K X$ for standard attributed graphs (§4).
- An empirical study on five inductive benchmarks under a strict inductive protocol, showing strong node-classification performance and leading link-prediction performance against representative deep graph SSL baselines (§5).
- A precise positioning against CCA-inspired graph SSL [9, 10]: GW-CCA inserts graph structure directly into the cross-covariance rather than using it as an augmentation-invariance loss (§3.3); Appendix C additionally contrasts GW-CCA with graph-regularised CCA [16–18], which uses Laplacian penalties.

## 2 Background

### 2.1 Inductive Attributed Graph Embedding

Let $G = (V, E)$ be a graph with $n = |V|$ nodes, adjacency matrix $A \in \mathbb{R}^{n \times n}$, and node feature matrix $X \in \mathbb{R}^{n \times p}$. Attributed graph embedding learns a representation $z^{(i)} \in \mathbb{R}^d$ for each node that captures both its attributes $x^{(i)}$ and its graph context. In the *inductive* setting, the embedding rule

must generalise to unseen nodes without learning a separate lookup vector for each training node: a method trains on a graph or training subgraph and later embeds new nodes using their features and observable incident edges, without retraining. This excludes purely transductive methods such as DeepWalk and node2vec [1, 2] unless an explicit out-of-sample extension is added.

### 2.2 Canonical Correlation Analysis

Given two paired feature matrices $X_1, X_2 \in \mathbb{R}^{n\times p}$ (the setting used by GW-CCA in §4, where both views share the same $p$-dimensional feature space), CCA learns linear projections $W, H \in \mathbb{R}^{p\times d}$ that maximise the cross-view cross-covariance

$$\arg\max_{W,H} \operatorname{Tr}(W^\top X_1^\top X_2 H), \quad \text{s.t.} \quad W^\top X_1^\top X_1 W = I_d,\ H^\top X_2^\top X_2 H = I_d; \tag{1}$$

see Hardoon et al. [19] for the derivation from the correlation coefficient. With $Z_1 = X_1 W$ and $Z_2 = X_2 H$, CCA projects paired points into representations whose cosine similarity ${z_1^{(i)}}^\top z_2^{(i)}$ is maximised (Figure 1a). Eq. 1 admits a closed-form solution via a singular value decomposition of a $p \times p$ matrix built from the covariances of $X_1$ and $X_2$; CCA is thus inductive and efficient when $p \ll n$. It captures only linear relationships and treats each same-index pair $(x_1^{(i)}, x_2^{(i)})$ symmetrically — discarding any additional pairing information such as graph similarity, which §3 addresses by inserting a graph similarity matrix $M$ into the cross-covariance of Eq. 1.

## 3 Graph-Weighted CCA

### 3.1 Objective

Standard CCA aligns paired points $(x_1^{(i)}, x_2^{(i)})$ under the assumption that same-index correspondence is the only pairing information. On attributed graphs we have richer pairing information — graph similarity $M_{ij}$ between nodes $v_i$ and $v_j$ — that CCA discards. GW-CCA generalises CCA by using $M$ to re-weight the cross-view alignment, so that structurally similar nodes are pulled together across views.

Concretely, given two views of data $X_1$ and $X_2$, GW-CCA learns a pair of projections $\widetilde{W}, \widetilde{H}$ by solving

$$\arg\max_{\widetilde{W},\widetilde{H}} \operatorname{Tr}(\widetilde{W}^\top X_1^\top M X_2 \widetilde{H}),\ \text{s.t.}\ \widetilde{W}^\top X_1^\top X_1 \widetilde{W} = I_d,\ \widetilde{H}^\top X_2^\top X_2 \widetilde{H} = I_d, \tag{2}$$

where $M \in \mathbb{R}^{n\times n}$ defines pairwise node similarity in the original graph, i.e., $M_{ij} = \operatorname{sim}(v_i, v_j)$. Two special cases are worth noting: setting $M = I_n$ recovers standard CCA [15] (Eq. 1); and, as we make precise in §4, our two-view construction $X_2 = \widehat{P}^K X$ reuses an SGC-style [12] propagation kernel, so an SGC-style spectral filter appears inside GW-CCA.

Writing the projected representations as $z_1^{(i)} = \widetilde{W}^\top x_1^{(i)}$ and $z_2^{(j)} = \widetilde{H}^\top x_2^{(j)}$ (Figure 1b), The objective of Eq. 2 admits an equivalent pointwise expansion over cross-view node pairs (Appendix G):

$$\operatorname{Tr}\left(\widetilde{W}^\top X_1^\top M X_2 \widetilde{H}\right) \;=\; \sum_{i,j=1}^{n} M_{ij}\, {z_1^{(i)}}^\top z_2^{(j)}, \tag{3}$$

This makes the role of $M$ explicit as a change to CCA's pairing structure: standard CCA is the $M = I_n$ case, which assigns weight 1 only to same-index pairs $(i, i)$ and weight 0 to all cross-node pairs. GW-CCA replaces that identity pairing with a graph-dependent soft pairing — node $i$ in view 1 aligns with node $j$ in view 2 whenever $M_{ij}$ is large — so structurally similar nodes are pulled together across views. The whitening constraints of Eq. 2 remain in force; Eq. 3 rewrites only the objective. After solving Eq. 2, we concatenate $X_1\widetilde{W}$ and $X_2\widetilde{H}$ to form the final $n \times 2d$ node representation matrix. The projections are linear in the features, but $M$ is a polynomial of the adjacency matrix (§4), so multi-hop graph structure enters the eigenvalue problem through $M$ — unlike standard CCA applied to the same two views, which would ignore $M$ entirely. How the two views $X_1, X_2$ and the similarity $M$ are constructed from a standard attributed graph with one feature matrix appears in §4.

### 3.2 Closed-Form Solution

Eq. 2 assumes $X_k^\top X_k$ is non-singular; when it is not (e.g., Yelp in §5), we solve the ridge-regularized variant obtained by replacing $X_k^\top X_k$ with $C_{kk} = X_k^\top X_k + \lambda_k I_p$ ($\lambda_k \geq 0$) in both the whitening constraints and the closed form below — setting $\lambda_k = 0$ recovers Eq. 2, and §5 shows GW-CCA is insensitive to $\lambda_1 = \lambda_2 = \lambda$. Let $T = X_1^\top M X_2$. A whitening change of variables reduces this (regularized) Eq. 2 to a truncated singular value decomposition (Appendix G):

$$C_{11}^{-1/2}\, T\, C_{22}^{-1/2} = U\, \Sigma\, V^\top, \qquad \widetilde{W} = C_{11}^{-1/2} U_{:,1:d}, \qquad \widetilde{H} = C_{22}^{-1/2} V_{:,1:d}, \tag{4}$$

where $U_{:,1:d}$ and $V_{:,1:d}$ hold the top-$d$ left and right singular vectors and $\Sigma_{ii}$ is the $i$-th singular value of the whitened graph-weighted cross-moment matrix — an $M$-weighted alignment score bounded by $\|M\|_2$. When $M = I_n$ (and features are centered) this reduces to the classical canonical correlation, bounded in $[0, 1]$; for general $M$ (in particular, non-symmetric row-stochastic $P$ on hub-heavy graphs) $\|M\|_2$ can exceed 1, so we refer to the $\Sigma_{ii}$ as $M$-weighted alignment scores rather than correlations. In practice $C_{kk}^{-1/2}$ is applied implicitly via a Cholesky factorisation of $C_{kk}$ — never formed as an explicit inverse — and only the top-$d$ singular triplets are computed. Three dense $p \times p$ matrices ($C_{11}$, $C_{22}$, and the cross term $T = X^\top (M X_2)$) plus the two Cholesky factors $L_1, L_2$ are materialised; the whitened SVD target $C_{11}^{-1/2} T C_{22}^{-1/2}$ itself is never formed explicitly, being accessed only through triangular solves against $L_1, L_2$. Concretely, the SVD is taken of $L_1^{-1} T\, L_2^{-\top}$ and the projections reconstructed via $\widetilde{W} = L_1^{-\top} U_{:,1:d}$, $\widetilde{H} = L_2^{-\top} V_{:,1:d}$, which yields a whitened solution differing from Eq. 4 by a rotation and identical singular values. An equivalent generalised eigenvalue formulation on $(T\, C_{22}^{-1}\, T^\top,\ C_{11})$ recovers the same $\widetilde{W}$ up to sign; we adopt the SVD form because it yields both projections from a single decomposition and is better conditioned than the generalised eigenvalue formulation.

### 3.3 Comparison to CCA-Inspired Graph Self-Supervised Learning

A more recent line of work uses CCA-style objectives as *losses* for training graph neural networks in a self-supervised manner [9, 10]. CCA-SSG [9] generates two augmented views of an input graph and trains a GNN encoder to (i) align the two views' representations and (ii) decorrelate feature dimensions, mirroring the invariance and whitening constraints of CCA. Graph Barlow Twins [10] adopts the closely related Barlow-Twins cross-correlation objective for the same purpose. GW-CCA differs from these methods in three ways. *(i) Shallow, closed-form solver vs. SGD.* We solve the CCA problem in closed form via SVD; there is no encoder to train, no augmentation schedule, and no learning-rate/epoch tuning. Consequently, GW-CCA is two to three orders of magnitude faster than SSL baselines in our experiments (Table 5), and its output is deterministic. *(ii) CCA as solver vs. CCA as loss.* CCA-SSG and G-BT use a CCA-inspired *loss* to train a neural encoder; GW-CCA *is* CCA, with the graph inserted into the cross-covariance and solved by eigen-decomposition. *(iii) Structural weighting vs. same-index invariance.* CCA-SSG/G-BT align the same node's representations across two augmented views (encouraging augmentation invariance). GW-CCA aligns *cross-node* pairs $(i, j)$ weighted by graph similarity $M_{ij}$ (Eq. 3), so it explicitly encourages structurally similar nodes—not just the same node under two random augmentations—to have similar cross-view representations.

Orthogonal to these graph-focused extensions, a separate line of work non-linearises CCA itself: Kernel CCA [19] maps inputs to a reproducing-kernel Hilbert space, Deep CCA [20] learns two deep neural encoders in place of the linear projections, and DCCAE [21] adds an autoencoder regulariser. GW-CCA differs on the opposite axis: it keeps the projections strictly linear but reweights the cross-covariance by graph similarity, so multi-hop graph structure enters through $M$ rather than through a deeper encoder — and the whole system remains closed-form, with no encoder training or augmentation schedule.

## 4 Instantiation for Attributed Graphs

Section 3 defines GW-CCA for arbitrary $(X_1, X_2, M)$; we now construct each from an attributed graph with feature matrix $X$ and adjacency $A$.

### 4.1 Graph-Aware View Construction

Attributed graphs come with a single feature matrix $X$; we set $X_1 = X$ and construct a second view $X_2 = f(X)$ deterministically from $X$ and the graph, using a graph-aware map $f : \mathbb{R}^{n\times p} \to \mathbb{R}^{n\times p}$. To incorporate graph information, we use the graph convolution of Wu et al. [12]:

$$X_2 = f(X) = \widehat{P}^K X, \tag{5}$$

where $\widehat{P} = (D+I_n)^{-1}(A+I_n)$ is a normalised adjacency matrix and $K$ is the number of propagation steps. As noted in §3.1, this is a row-stochastic variant of SGC's [12] propagation kernel: the $i$-th row of $X_2$ is a $K$-hop weighted aggregate of neighbouring features, so the second view carries local graph topology into the feature space, and CCA on $(X, \widehat{P}^K X)$ becomes a graph-aware alignment problem. Computing $X_2$ takes $\mathcal{O}(K|E|p)$ time when $\widehat{P}$ is sparse. The propagation kernel $\widehat{P}$ and the similarity kernel used in §4.2 play distinct roles in the objective. $\widehat{P}$ smooths features into a second view $X_2 = \widehat{P}^K X$ that CCA aligns with $X_1 = X$ (row normalisation with self-loops, for SGC-style low-pass filtering), while $P = D^{-1}A$ (without self-loops) defines a pair-weight matrix $M$ that reweights which cross-view row pairs contribute to Eq. 2. Both kernels are low-order polynomials of row-normalised adjacencies and act as low-pass filters emphasising neighbourhood-smooth feature components; GW-CCA then selects projections whose graph-filtered cross-view alignment is largest — the same low-pass regime that motivates SGC [12], now driving the CCA objective rather than a supervised loss.

### 4.2 Random-Walk Pair Weights

We use the truncated random-walk similarity

$$M = \frac{P + P^2}{2}, \qquad P = D^{-1}A, \tag{6}$$

which encodes two deliberate design decisions.

**Row-normalised, not symmetric-normalised.** We take $P = D^{-1}A$ rather than the self-looped $\widehat{P}$ used for the second view because $[P^r]_{ij}$ is a proper transition probability — the probability that an $r$-step random walk from $v_i$ ends at $v_j$. Symmetric normalisation mixes this with degree factors and loses the random-walk interpretation that motivates using $M$ as a similarity in the first place. The two kernels' complementary roles play together in §4.3. A symmetric normalisation could also be used when a bounded spectral norm is desired, but it loses the direct random-walk interpretation. We use the row-stochastic form throughout because $M$ is intended to encode transition-based proximity.

**Truncation at $r = 2$.** We choose $T = 2$ because the GW-CCA objective uses $M$ only to weight cross-view pairs: first-order (direct-neighbour) proximity aligns connected nodes, and second-order (shared-neighbour) proximity aligns nodes with overlapping neighbourhoods — both are informative for downstream tasks. Higher powers $P^r$ spread mass over increasingly distant nodes and can blur class-relevant local structure, at additional propagation cost. This is consistent with LINE [3], which explicitly models the same two orders, and with DeepWalk's implicit target: Qiu et al. [4] show DeepWalk factorises $\log\big(\mathrm{vol}(G) \cdot \frac{1}{T}\sum_{r=1}^{T} P^r \cdot D^{-1}\big)$, so $T = 2$ (dropping the log/degree scaling) recovers $M$ up to a constant. §5 confirms that raising $T$ beyond 2 does not improve downstream performance. Although $P^2$ can be much denser than $P$ in graphs with high-degree hubs, GW-CCA never materialises $M$ explicitly; it applies $M$ to $X_2$ via two sparse SpMV rounds, $MX_2 = \frac{1}{2}(PX_2 + P(PX_2))$, at $\mathcal{O}(|E|p)$ cost regardless of $P^2$'s density — the property the complexity analysis in §4.5 actually relies on.

### 4.3 Special Cases and Ablations

Table 1 places GW-CCA inside the design space $(X_1, X_2, M)$. Setting $M = I_n$ recovers standard CCA on the chosen two views; combining $M = I_n$ with the propagated view $X_2 = \widehat{P}^K X$ yields the CCA* baseline reported in Tables 3–4, which is the cleanest ablation of GW-CCA: everything except the graph-weighted cross-covariance is held fixed, so any margin of GW-CCA over CCA* is attributable to $M$ alone.

**Table 1:** Special cases of Eq. 2 on the two-view construction $(X, \widehat{P}^K X)$. CCA$^*$ is the graph-unweighted counterpart of GW-CCA; GW-CCA replaces the ordinary cross-covariance with a graph-weighted one.

| $X_1$ | $X_2$ | $M$ | Method |
|---|---|---|---|
| $X$ | $\widehat{P}^K X$ | $I_n$ | CCA$^*$ (Table 3 baseline) |
| $X$ | $\widehat{P}^K X$ | $(P + P^2)/2$ | **GW-CCA** |

**Algorithm 1** GW-CCA: training and inductive inference

**Training phase.** Inputs: training-subgraph adjacency $A$, training features $X \in \mathbb{R}^{n \times p}$, embedding dim $d$, propagation depth $K$, ridge $\lambda \geq 0$. Returns $\widetilde{W}, \widetilde{H} \in \mathbb{R}^{p \times d}$.

$P \leftarrow D^{-1}A; \quad \widehat{P} \leftarrow (D + I_n)^{-1}(A + I_n)$

$X_2 \leftarrow \widehat{P}^K X$ ▷ $K$ sparse SpMVs; do not materialise $\widehat{P}^K$

$MX_2 \leftarrow \frac{1}{2}\big(PX_2 + P(PX_2)\big)$ ▷ apply $M = (P + P^2)/2$ to $X_2$ via two sparse SpMV rounds ($\mathcal{O}(|E|p)$); do not materialise $M$ or $P^2$

$C_{11} \leftarrow X^\top X + \lambda I_p;\ C_{22} \leftarrow X_2^\top X_2 + \lambda I_p;\ T \leftarrow X^\top (MX_2)$

Cholesky factorise $C_{kk} = L_k L_k^\top$ for $k \in \{1, 2\}$, giving $C_{kk}^{-1/2}$ implicitly

$U, \Sigma, V \leftarrow \textsc{TruncatedSVD}\Big(C_{11}^{-1/2}\, T\, C_{22}^{-1/2},\ \text{rank} = d\Big)$

$\widetilde{W} \leftarrow C_{11}^{-1/2} U;\ \widetilde{H} \leftarrow C_{22}^{-1/2} V$ ▷ Eq. 4

**Inductive inference.** Given the full observed graph $A'$ and features $X'$ — including previously unseen test nodes and their incident edges, but *no test labels* — return $Z' \in \mathbb{R}^{n' \times 2d}$ for every node of $A'$:

$\widehat{P}' \leftarrow (D' + I_{n'})^{-1}(A' + I_{n'})$

$X'_2 \leftarrow \widehat{P}'^K X'$ ▷ uses test-node features and edges only at inference, after $\widetilde{W}, \widetilde{H}$ are fixed

**return** $Z' \leftarrow [\, X' \widetilde{W} \,\|\, X'_2 \widetilde{H} \,]$

### 4.4 Algorithm and Inductive Inference

Algorithm 1 lays out both phases in a single block. Training uses only the training-subgraph adjacency $A$ (edges among training nodes) to learn the projection pair $(\widetilde{W}, \widetilde{H})$; inductive inference then applies them to the full observed graph $(A', X')$ — containing $(A, X)$ as its training-node-induced subgraph plus previously unseen test nodes and any edges incident to them — without retraining or fine-tuning. This matches the standard inductive protocol of Hamilton et al. [22]: the graph structure among test nodes is observable at inference time, but their labels are not, and no test-node features or edges enter training.

**No materialisation of graph powers.** All graph powers in Algorithm 1 are applied by repeated sparse matrix–vector multiplications; neither $\widehat{P}^K$, $P^2$, nor $M$ is formed explicitly. This is what enables the $\mathcal{O}(|E|p)$ propagation cost analysed in §4.5.

**Determinism.** For fixed inputs $(A, X, K, d, \lambda)$, Algorithm 1 is deterministic up to numerical-library nondeterminism (BLAS/LAPACK implementation, thread scheduling, sparse-multiplication ordering) and sign ambiguities in singular vectors, and provided that the top-$d$ singular values of $C_{11}^{-1/2} T C_{22}^{-1/2}$ are separated from the rest. Unlike deep SSL, there is no random seed to sweep, no augmentation sampling, and no variance to report.

### 4.5 Complexity

Training GW-CCA costs $\mathcal{O}(K|E|p + np^2 + p^3)$, linear in $|E|$ and $n$; inductive inference on a graph with $n'$ nodes and $|E'|$ edges costs $\mathcal{O}(K|E'|p + n'pd)$, with no retraining or fine-tuning. GNN baselines with $L$ layers and hidden dimension $h$ cost $\mathcal{O}(L|E|h)$ per epoch across 50–200 epochs — $10^2$–$10^4$ more sparse-matrix multiplications than GW-CCA, matching the orders-of-magnitude

**Table 2:** Dataset statistics. Node-inductive datasets use their default realistic split; PPI is graph-inductive (24 disjoint protein–protein-interaction graphs, split 20/2/2).

| Dataset | Task | # Nodes | # Edges | # Features | # Classes | Train/Val/Test |
|---|---|---|---|---|---|---|
| Reddit | Node-inductive | 231,443 | 11,606,919 | 602 | 41 | 0.66/0.10/0.24 |
| ogbn-arxiv | Node-inductive | 169,343 | 1,166,243 | 128 | 40 | 0.54/0.18/0.28 |
| Yelp | Node-inductive | 716,847 | 6,997,410 | 300 | 100* | 0.75/0.15/0.10 |
| ogbn-products | Node-inductive | 2,449,029 | 61,859,140 | 100 | 47 | 0.08/0.02/0.90 |
| PPI | Graph-inductive | 56,944 | 806,928 | 50 | 121* | 20/2/2 graphs |

∗ Yelp and PPI are multi-label classification datasets.

speedup measured in Table 5. Appendix F gives the per-line derivation, the exact $(K+2)|E|p$ constant, and the memory analysis.

# 5 Experiments

We empirically evaluate the quality of representations produced by GW-CCA on node classification and link prediction. All experiments are conducted in a fully inductive, unsupervised manner.

## 5.1 Datasets

We use five inductive benchmarks (Table 2): four node-inductive graphs — Reddit [22], ogbn-arxiv [23], Yelp [24], and ogbn-products [23] at 2.4M nodes — and one graph-inductive benchmark, PPI [22], whose 24 disjoint protein–protein-interaction graphs are split 20/2/2 between train, val, and test. All splits are the datasets' default realistic partitions. Detailed descriptions and download URLs appear in Appendix A.

## 5.2 Baselines

We compare against ten baselines chosen to cover the major methodological families of inductive graph representation learning (2017–2023), so that GW-CCA is compared against a strong exemplar of each paradigm rather than duplicated instances of one: one edge-prediction self-supervised message-passing network, **GraphSAGE** [22] (the unsupervised variant, trained with the link-prediction BCE pretext of the original paper); four contrastive / mutual-information methods, **DGI** [7], **GRACE** [5], **BGRL** [6], and **GGD** [25]; two CCA-inspired SSL methods, **CCA-SSG** [9] and **GraphACL** [26]; one masked-autoencoder method, **GraphMAE2** [11]; one linear propagation baseline, **SGC** [12]; and the **Raw+LR** control (raw features fed directly into the logistic-regression probe). We also report the CCA* ablation of GW-CCA (Table 1, $M = I_n$), which isolates the effect of the graph-weighted cross-covariance.

**Strict inductive protocol.** To keep the comparison apples-to-apples, we re-run every baseline under a strict inductive protocol: each encoder is trained on the training-node-induced subgraph $A^{\text{tr}}$ only, then frozen and applied to the full graph $A'$ for inference (see §4.4). This is stricter than some prior papers, which train on the full graph and merely hold out test-node labels; consequently the numbers in Tables 3–4 are lower than the transductive-shortcut numbers those papers report. All deep baselines run their official PyTorch implementations, mini-batched with the neighbour-subsampling scheme of Hamilton et al. [22]. Full hyperparameter grids appear in Appendix B.

## 5.3 Experimental Setup

Across all datasets we set $K = 3$ and $M = (P + P^2)/2$; higher truncation orders do not improve performance. We choose $d = 120$ for Reddit and ogbn-arxiv, $d = 300$ for Yelp, $d = 100$ for ogbn-products, and $d = 50$ for PPI. Because the covariance matrices $X_1^\top X_1$ and $X_2^\top X_2$ of Yelp are singular, we set $\lambda_1 = \lambda_2 = \lambda = 0.01$ on Yelp; on the other datasets $\lambda = 0$. GW-CCA is empirically insensitive to $d$, $K$, and $\lambda$ near the reported values across all datasets; the full sensitivity analysis appears in Appendix H. Fairness details — matched splits, shared LBFGS probe, edge splits, and per-method hyperparameter treatment — are documented in Appendix B, where the setup is conservative toward GW-CCA.

**Table 3:** Inductive node classification (Micro-F1, %; mean ± std over 5 seeds for deep baselines, single value for deterministic methods). Best per column in bold.

| Method | ogbn-arxiv | Reddit | Yelp | ogbn-products | PPI |
|---|---|---|---|---|---|
| Raw features + LR | 52.5 | 68.0 | 48.6 | 47.2 | 43.2 |
| SGC [12] | 69.5 | 94.4 | 52.9 | 73.0 | 45.7 |
| GraphSAGE [22] | 66.1±0.2 | 92.7±1.4 | 56.9±0.5 | 69.4±0.3 | **62.8±0.3** |
| DGI [7] | 61.6±4.2 | 91.5±4.3 | 52.8±4.8 | 69.2±0.4 | 59.9±1.5 |
| GRACE [5] | 65.9±0.4 | 90.8±0.5 | 55.7±0.3 | 73.3±0.3 | 61.8±0.2 |
| GGD [25] | 68.5±0.2 | 93.2±0.9 | 52.0±0.3 | 67.1±0.4 | 57.4±0.5 |
| CCA-SSG [9] | 68.8±0.3 | 92.4±1.0 | 50.0±0.3 | 54.7±1.3 | 58.5±0.1 |
| BGRL [6] | 68.9±0.2 | 91.1±1.4 | 56.0±0.5 | 71.8±0.3 | 58.2±0.2 |
| GraphACL [26] | 69.2±0.2 | 90.3±0.6 | 53.5±0.4 | 64.3±0.3 | 56.3±0.3 |
| GraphMAE2 [11] | 68.7±0.3 | 94.6±0.8 | 57.1±0.4 | **74.3±0.3** | 61.3±0.8 |
| CCA* | 69.5 | 95.4 | 49.7 | 73.0 | 48.2 |
| **GW-CCA** | **69.7** | **96.3** | **61.1** | 72.9 | 55.3 |

**Seeds.** Deep baselines are averaged over five seeds; tables report mean ± std. GW-CCA, CCA*, SGC, and Raw+LR are deterministic. When comparing against stochastic baselines, we treat a method as meaningfully ahead when its mean exceeds the other by more than the stochastic side's reported standard deviation, and as statistically comparable otherwise.

### 5.4 Node Classification

**Metrics.** Following Hamilton et al. [22], we train a logistic-regression classifier on training-node representations and evaluate on test nodes. We use a GPU LBFGS probe (equivalent to `sklearn`'s `LogisticRegression(solver="lbfgs", C=1.0)` but 13–60× faster; Appendix B), which is deterministic given the fixed seed and converges to the global optimum on the convex objective. Micro-F1 is reported for multi-label datasets (Yelp, PPI) and accuracy = Micro-F1 for single-label ones.

**Results.** Table 3 shows GW-CCA leads 3 of 5 datasets on Micro-F1 (ogbn-arxiv, Reddit, Yelp), stays within 1.4 pp of GraphMAE2 on ogbn-products, and trails GraphSAGE on PPI — an expected weak spot on 24 small graphs ( 2.4K nodes each) that favour multi-layer message passing. On Yelp, GW-CCA leads GraphMAE2 by +4.0 pp (61.1 vs 57.1) and its graph-unweighted ablation CCA* by +11.4 pp (61.1 vs 49.7). CCA* is competitive on single-label datasets; the remaining margin of GW-CCA over CCA* (+0.9 pp Reddit, +11.4 pp Yelp, +7.1 pp PPI) isolates the contribution of $M$.

### 5.5 Link Prediction

**Metrics.** We adopt the strict-inductive link-prediction protocol: from the graph edges between two test nodes, we hold out 20% as positive test edges and sample an equal number of test–test non-edges as negatives (fixed seed shared across all methods and seeds). The encoder is applied to the graph with the held-out positives removed to produce test-node embeddings. A logistic-regression edge classifier is trained on Hadamard products of endpoint embeddings from an equal-sized set of train–train edges (positives) and train–train non-edges (negatives), then applied to the held-out test edges. Full protocol specifics — which edges are preserved at inference, negative sampling, and hyperparameter selection — appear in Appendix D. We report AUC in Table 4 and average precision (AP) in Appendix E (AP tracks AUC closely).

**Results.** Table 4 shows GW-CCA leads AUC on *all five* datasets, with margins from +0.7 pp on Yelp (over GRACE) to +2.7 pp on Reddit (over GraphSAGE). The gap over CCA* (the graph-unweighted ablation) is substantial across datasets: +15.5 pp AUC on PPI, +7.4 pp on Yelp, +2.5 pp on Reddit — supporting $M$'s role as a pairwise-similarity structure. Link prediction is a pairwise-similarity task and directly benefits from the pairwise structure $M_{ij}$ encodes, while node classification only needs per-class discrimination.

**Table 4:** Inductive link prediction (AUC, %; mean ± std over 5 seeds for deep baselines, single value for deterministic methods). Best per column in bold. Average-precision (AP) tracks AUC closely and appears in Appendix E.

| Method | ogbn-arxiv | Reddit | Yelp | ogbn-products | PPI |
|---|---|---|---|---|---|
| Raw features + LR | 79.9 | 76.4 | 73.2 | 83.1 | 49.7 |
| SGC [12] | 90.6 | 94.2 | 85.4 | 97.8 | 75.0 |
| GraphSAGE [22] | 95.2±0.1 | 94.9±0.9 | 87.5±0.4 | 97.0±0.2 | 83.4±0.2 |
| DGI [7] | 88.3±2.6 | 89.7±2.2 | 92.6±2.2 | 93.1±0.9 | 86.8±0.4 |
| GRACE [5] | 92.5±0.5 | 93.5±0.4 | 93.0±0.3 | 96.8±0.1 | 87.5±0.2 |
| GGD [25] | 95.2±0.2 | 93.1±1.0 | 92.7±0.0 | 98.1±0.1 | 86.1±0.3 |
| CCA-SSG [9] | 94.2±0.1 | 92.6±0.8 | 92.3±0.1 | 84.3±1.2 | 85.5±0.2 |
| BGRL [6] | 93.2±0.9 | 92.9±0.9 | 92.5±0.2 | 87.5±0.6 | 83.0±0.2 |
| GraphACL [26] | 95.3±0.1 | 91.2±0.8 | 92.2±0.2 | 97.0±0.2 | 76.6±0.8 |
| GraphMAE2 [11] | 86.0±1.8 | 88.4±0.7 | 91.1±0.2 | 97.0±0.4 | 81.3±0.9 |
| CCA* | 94.7 | 95.1 | 86.3 | 97.6 | 73.7 |
| **GW-CCA** | **97.5** | **97.6** | **93.7** | **99.5** | **89.2** |

**Table 5:** Encoder training wall-clock (seconds; mean ± std over 5 seeds for deep baselines, single value for deterministic methods). GW-CCA and CCA* run on CPU (no GPU); deep baselines run on a single A10G GPU.

| Method | ogbn-arxiv | Reddit | Yelp | ogbn-products | PPI |
|---|---|---|---|---|---|
| GraphSAGE [22] | 1537±76 | 2163±58 | 279±4 | 207±3 | 1888±22 |
| DGI [7] | 576±24 | 289±5 | 306±7 | 181±30 | 357±1 |
| GRACE [5] | 842±37 | 497±25 | 826±40 | 289±3 | 525±23 |
| GGD [25] | 29±0.3 | 167±0.1 | 226±3 | 148±2 | 15±0.2 |
| CCA-SSG [9] | 11±0.1 | 1703±35 | 577±25 | 387±17 | 11±0.2 |
| BGRL [6] | 43±0.1 | 2053±59 | 723±14 | 379±6 | 38±0.4 |
| GraphACL [26] | 141±0.4 | 125±0.2 | 969±3 | 139±0.1 | 117±0.3 |
| GraphMAE2 [11] | 40±0.0 | 1425±6 | 1665±4 | 599±3 | 238±3 |
| CCA* | 1.5 | 35.7 | 18.1 | 7.0 | 0.2 |
| **GW-CCA** | **1.8** | **42.0** | **28.1** | **12.1** | **0.3** |

### 5.6 Efficiency

GW-CCA runs on CPU in NumPy/SciPy; deep baselines run on a single A10G GPU with each method's reference-paper default configuration (see Appendix B). GW-CCA is orders of magnitude faster — e.g., $854\times$ over GraphSAGE on ogbn-arxiv ($1537 \rightarrow 1.8$ s), $41\times$ over CCA-SSG on Reddit ($1703 \rightarrow 42$ s), and $29\times$ over GRACE on Yelp ($826 \rightarrow 28$ s); on the smallest graph (PPI) the gap widens to $6000\times$ over GraphSAGE ($1888 \rightarrow 0.3$ s). This CPU-only, closed-form profile suits deployment settings that GPU-trained encoders struggle with: hourly-updating recommender or ads-ranking graphs, CPU-only infrastructure, and hyperparameter-free retraining that eliminates the seed sweeps and grid searches driving deep-SSL iteration time.

## 6 Conclusion

We proposed GW-CCA, a shallow inductive attributed graph embedding method that incorporates graph information into canonical correlation analysis. GW-CCA learns a pair of projections rather than per-node embeddings, making it naturally inductive. On five inductive benchmarks under a strict inductive protocol, it outperforms state-of-the-art deep SSL methods (2017–2023) on link prediction across all five datasets and is competitive or best on node classification, while being orders of magnitude faster. Interesting directions include extending GW-CCA to multi-graph datasets and exploring richer attribute mappings.

# References


[1] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014. 1, 3

[2] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016. 1, 3

[3] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on world wide web*, pages 1067–1077, 2015. 1, 5

[4] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, pages 459–467, 2018. 1, 5

[5] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. Deep graph contrastive representation learning. *arXiv preprint arXiv:2006.04131*, 2020. 1, 7, 8, 9, 12, 14

[6] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Mehdi Azabou, Eva L. Dyer, Rémi Munos, Petar Veličković, and Michal Valko. Large-scale representation learning on graphs via bootstrapping. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 7, 8, 9, 12, 14

[7] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep graph infomax. 2019. 1, 7, 8, 9, 12, 14

[8] Zhen Peng, Wenbing Huang, Minnan Luo, Qinghua Zheng, Yu Rong, Tingyang Xu, and Junzhou Huang. Graph representation learning via graphical mutual information maximization. In *Proceedings of The Web Conference 2020*, pages 259–270, 2020. 1

[9] Hengrui Zhang, Qitian Wu, Junchi Yan, David Wipf, and Philip S. Yu. From canonical correlation analysis to self-supervised graph neural networks. In *Advances in Neural Information Processing Systems*, 2021. 1, 2, 4, 7, 8, 9, 12, 14

[10] Piotr Bielak, Tomasz Kajdanowicz, and Nitesh V. Chawla. Graph Barlow Twins: A self-supervised representation learning framework for graphs. *Knowledge-Based Systems*, 256: 109631, 2022. 1, 2, 4

[11] Zhenyu Hou, Yufei He, Yukuo Cen, Xiao Liu, Yuxiao Dong, Evgeny Kharlamov, and Jie Tang. Graphmae2: A decoding-enhanced masked self-supervised graph learner. In *Proceedings of the ACM Web Conference (WWW)*, 2023. 1, 7, 8, 9, 12, 14

[12] Felix Wu, Tianyi Zhang, Amauri Holanda de Souza Jr, Christopher Fifty, Tao Yu, and Kilian Q Weinberger. Simplifying graph convolutional networks. *arXiv preprint arXiv:1902.07153*, 2019. 1, 2, 3, 5, 7, 8, 9, 12, 14

[13] Fabrizio Frasca, Emanuele Rossi, Davide Eynard, Ben Chamberlain, Michael Bronstein, and Federico Monti. SIGN: Scalable inception graph neural networks. In *ICML Workshop on Graph Representation Learning and Beyond*, 2020. 1

[14] Hao Zhu and Piotr Koniusz. Simple spectral graph convolution. In *International Conference on Learning Representations (ICLR)*, 2021. 1

[15] Harold Hotelling. Relations between two sets of variates. In *Breakthroughs in statistics*, pages 162–190. Springer, 1992. 2, 3

[16] Yun-Hao Yuan and Quan-Sen Sun. Graph regularized multiset canonical correlations with applications to joint feature extraction. *Pattern Recognition*, 47(12):3907–3919, 2014. 2, 13

[17] Jia Chen, Gang Wang, Yanning Shen, and Georgios B Giannakis. Canonical correlation analysis with common graph priors. In *2018 IEEE Statistical Signal Processing Workshop (SSP)*, pages 488–492. IEEE, 2018. 13

[18] Jia Chen, Gang Wang, Yanning Shen, and Georgios B Giannakis. Canonical correlation analysis of datasets with a common source graph. *IEEE Transactions on Signal Processing*, 66(16): 4398–4408, 2018. 2, 13

[19] David R Hardoon, Sandor Szedmak, and John Shawe-Taylor. Canonical correlation analysis: An overview with application to learning methods. *Neural computation*, 16(12):2639–2664, 2004. 3, 4

[20] Galen Andrew, Raman Arora, Jeff Bilmes, and Karen Livescu. Deep canonical correlation analysis. In *International conference on machine learning*, pages 1247–1255. PMLR, 2013. 4

[21] Weiran Wang, Raman Arora, Karen Livescu, and Jeff Bilmes. On deep multi-view representation learning. In *International conference on machine learning*, pages 1083–1092. PMLR, 2015. 4

[22] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017. 6, 7, 8, 9, 11, 12, 14

[23] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *arXiv preprint arXiv:2005.00687*, 2020. 7, 11, 12

[24] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. Graphsaint: Graph sampling based inductive learning method. *arXiv preprint arXiv:1907.04931*, 2019. 7, 11

[25] Yizhen Zheng, Shirui Pan, Vincent CS Lee, Yu Zheng, and Philip S Yu. Rethinking and scaling up graph contrastive learning: An extremely efficient approach with group discrimination. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 7, 8, 9, 12, 14

[26] Teng Xiao, Huaisheng Zhu, Zhengyu Chen, and Suhang Wang. Simple and asymmetric graph contrastive learning without augmentations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 7, 8, 9, 12, 14

[27] Radim Rehurek and Petr Sojka. Software framework for topic modelling with large corpora. In *In Proceedings of the LREC 2010 workshop on new challenges for NLP frameworks*. Citeseer, 2010. 11

[28] Kuansan Wang, Zhihong Shen, Chiyuan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. Microsoft academic graph: When experts are not enough. *Quantitative Science Studies*, 1(1):396–413, 2020. 11

[29] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. 12

[30] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256, 2010. 12

[31] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 12

# A Dataset Details

**Reddit [22].** The Reddit graph is constructed from Reddit posts made in September 2014. Each node is a post; two posts are connected if the same user commented on both. Following the inductive setup of Hamilton et al. [22], the first 20 days of the month are used for training and the rest for testing (with 30% for validation). Features are 602-dimensional: the average of 300-dimensional GloVe CommonCrawl word vectors [27] over the post's words, concatenated with post-level statistics (score, comment counts, etc.), following Hamilton et al. [22].

**ogbn-arxiv [23].** ogbn-arxiv is a directed citation graph over Computer Science ArXiv papers indexed by the Microsoft Academic Graph [28]. Each paper's feature vector is the average Word2Vec embedding of the words in its title and abstract. The temporal split follows Reddit.

**Yelp [24].** The Yelp dataset is generated from businesses, reviews, and user data on the Yelp Open Challenge. Nodes are users and edges are user friendship links. Features are 300-dimensional aggregated Word2Vec embeddings pre-trained on GoogleNews.

**ogbn-products [23].** ogbn-products is the Amazon product co-purchasing network from the Open Graph Benchmark: 2.4M product nodes, 61.9M undirected edges, 100-dimensional bag-of-words features reduced by PCA, and 47 product categories. We use the official OGB split (0.08 / 0.02 / 0.90 for train / val / test), which is sales-rank-based rather than random.

**PPI [22].** The protein–protein interaction (PPI) benchmark is 24 disjoint graphs from different human tissues, split 20 / 2 / 2 into train / val / test (fully graph-inductive: test nodes and edges are in graphs never seen at training time). Each node is a protein with 50 positional / motif features; each of 121 protein functions is a separate binary label (multi-label). This is the canonical graph-inductive benchmark introduced with GraphSAGE.

# B Baseline Implementations and Hyperparameter Search

**Code sources.** Every baseline runs either the authors' official open-source release or the canonical implementation shipped with PyTorch Geometric (PyG) [29]; we do not reimplement any encoder ourselves, except SGC which is five lines of NumPy following Wu et al. [12] Eq. 11. Table 6 lists every method with its reference and code source verbatim so results can be reproduced exactly.

**Table 6:** Baseline lineup: reference and code source. All implementations are the authors' official release or a canonical PyG / DGL port.

| Method | Year / Venue | Reference | Code source |
|---|---|---|---|
| Raw + LR | — | — | `scikit-learn` `LogisticRegression`† |
| SGC | 2019 / ICML | Wu et al. [12] | 5 lines of NumPy (paper Eq. 11) |
| GraphSAGE | 2017 / NeurIPS | Hamilton et al. [22] | PyG `SAGEConv` (canonical) |
| DGI | 2019 / ICLR | Velickovic et al. [7] | PyG `DeepGraphInfomax` (canonical) |
| GRACE | 2020 / ICML SSL | Zhu et al. [5] | Official `CRIPAC-DIG/GRACE`, model classes verbatim |
| GGD | 2022 / NeurIPS | Zheng et al. [25] | Official code, model classes verbatim |
| CCA-SSG | 2021 / NeurIPS | Zhang et al. [9] | PyG port of the authors' official DGL code |
| BGRL | 2022 / ICLR | Thakoor et al. [6] | Official `BGRL_Pytorch/models.py` |
| GraphACL | 2023 / NeurIPS | Xiao et al. [26] | Official code |
| GraphMAE2 | 2023 / WWW | Hou et al. [11] | Official code |

† We use a torch `LBFGS` probe on GPU for all methods' downstream classifier evaluation; it is algorithmically equivalent to `sklearn`'s `LogisticRegression(solver="lbfgs", C=1.0)` but 13–60× faster and bit-deterministic given a fixed seed.

**Hyperparameters.** For each deep baseline we use the default training configuration reported in the reference paper (hidden dimension, layers, learning rate, epochs, augmentation rates, temperature, mask ratio, EMA decay, and so on). Where defaults produced a clearly under-trained encoder on our benchmarks — loss plateauing within the first 10 epochs, or downstream accuracy falling below the Raw+LR control — we grid-searched learning rate $\in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, hidden dimension $\in \{128, 256, 512\}$, and number of layers $\in \{2, 3, 4\}$, together with method-specific hyperparameters (dropout, augmentation / edge-drop / feature-mask rate, EMA decay, InfoNCE temperature, mask ratio) explored around their reference-paper defaults; all remaining hyperparameters were kept at the reference defaults. All deep baselines use Glorot initialisation [30] and Adam [31]. For graphs whose full adjacency exceeds GPU memory we mini-batch with the neighbourhood-subsampling scheme of Hamilton et al. [22]. Following the GraphSAGE paper's unsupervised setup, GraphSAGE runs one epoch over random-walk data (50 walks of length 5 per node). Note that GW-CCA itself is not tuned per dataset: the embedding dimension $d \in \{50, 100, 120, 300\}$ tracks the feature dimensionality of each benchmark (final values in §5); $K = 3$ is fixed across all datasets; $\lambda$ is fixed at 0 except on Yelp where $\lambda = 0.01$ is required for numerical stability. This makes the setup conservative toward GW-CCA — baselines are permitted small per-dataset LR / hidden-dim adjustments where their defaults were under-training, whereas GW-CCA uses a single triple $(d, K, \lambda)$ per dataset with $d$ chosen from the small set $\{50, 100, 120, 300\}$ without per-dataset tuning, and Appendix H confirms insensitivity to the exact values.

**Feature centering ablation.** Classical CCA operates on column-centered features. GW-CCA as reported in the main tables uses raw feature matrices; the training-subgraph column mean $\mu_X$ can equivalently be subtracted from all node features before running Algorithm 1, following the sklearn

CCA convention (extended to the inductive setting by freezing $\mu_X$ at training and applying the same shift to new nodes at inference). Table 7 compares the two configurations (single seed, seed$= 42$) across all five datasets and both tasks. The two settings agree to within roughly one point on every dataset and metric, with the uncentered variant slightly ahead on the mean ($\Delta = -0.34$ NC accuracy, $-0.27$ LP-AUC, $-0.29$ LP-AP averaged over datasets; uncentered wins 4/5 on NC). Residual global mean shifts in the learned embeddings are absorbed by the LBFGS probe's bias term, so the two are close as expected. We report uncentered results as the default because they are marginally better on the mean and match our released code.

Intuitively, centering barely moves the needle because three mechanisms progressively damp the rank-one correction: (i) the uncentered $X_k^\top X_k$ differs from the centered covariance only by $n\mu_k\mu_k^\top$, and whitening by $C_{kk}^{-1/2}$ shrinks this rank-one term to a small perturbation of the whitened basis; (ii) the L2 row-normalisation of the final embeddings $Z_k$ at inference further attenuates residual scale differences; and (iii) any remaining global mean shift is absorbed by the LBFGS probe's bias term. Centering matters most *before* whitening in the classical CCA pipeline; here, the compound effect of whitening + row-norm + probe bias renders the choice largely inert.

**Table 7:** GW-CCA with and without training-subgraph feature centering (seed$= 42$). NC = node-classification accuracy (%); LP-AUC and LP-AP are computed on validation edges. $\Delta$ = centered $-$ raw.

| | NC | | | LP-AUC | | | LP-AP | | |
|---|---|---|---|---|---|---|---|---|---|
| Dataset | raw | cent. | $\Delta$ | raw | cent. | $\Delta$ | raw | cent. | $\Delta$ |
| ogbn-arxiv | 69.74 | 69.78 | $+0.04$ | 97.50 | 97.74 | $+0.23$ | 97.71 | 97.88 | $+0.17$ |
| Reddit | 96.30 | 96.24 | $-0.07$ | 97.56 | 96.85 | $-0.70$ | 97.19 | 96.52 | $-0.67$ |
| Yelp | 61.10 | 60.16 | $-0.93$ | 93.75 | 94.08 | $+0.34$ | 95.35 | 95.53 | $+0.17$ |
| ogbn-products | 72.89 | 72.52 | $-0.38$ | 99.55 | 99.55 | $+0.00$ | 99.62 | 99.62 | $+0.00$ |
| PPI | 55.26 | 54.88 | $-0.38$ | 89.21 | 88.01 | $-1.21$ | 89.52 | 88.38 | $-1.14$ |
| mean $\Delta$ | | $-0.34$ | | | $-0.27$ | | | $-0.29$ | |

## C Comparison to Graph-Regularised CCA

A parallel line of work also brings graph information into CCA [16–18], but does so as a regulariser on the projections rather than through the correlation objective. A representative graph-regularised CCA solves

$$\max \operatorname{Tr}(W^\top X_1^\top X_2 H) - \gamma_1 \operatorname{Tr}(W^\top X_1^\top L X_1 W) - \gamma_2 \operatorname{Tr}(H^\top X_2^\top L X_2 H), \tag{7}$$

where $L$ is a graph Laplacian and $\gamma_1, \gamma_2 \geq 0$. This formulation is *not* a special case of ours. Writing $Z_k = X_k \cdot (\text{proj})$ for view $k$, the Laplacian penalty pulls the *same-view* representations of graph neighbours together (each term has the form $\operatorname{Tr}(Z_k^\top L Z_k)$), so the graph is used to smooth each view individually. In contrast, GW-CCA replaces the cross-covariance $X_1^\top X_2$ with $X_1^\top M X_2$ (Eq. 2), which is equivalent to a $M$-weighted alignment objective on the *cross-view* pairs (Eq. 3): graph structure directly selects which pairs $(i, j)$ should be aligned across views rather than smoothing each view against itself.

The three lines of work also differ in purpose, view construction, and scale. Chen et al. [17, 18] target graph signal denoising and estimation with pre-existing graph priors, and Yuan and Sun [16] targets joint feature extraction for multi-view classification (e.g., face recognition); none of them target inductive attributed graph embedding. They also assume that two views $X_1, X_2$ are given by the problem, whereas GW-CCA constructs $X_2 = \widehat{P}^K X$ from a single attribute matrix so that it applies to standard attributed graphs with only one feature view. Finally, prior graph-regularised CCA has been validated on datasets with fewer than $10^3$ samples, while we operate on graphs with $10^5$–$10^6$ nodes.

## D Link-Prediction Protocol Specifics

Full specification of the link-prediction protocol used in §5 and Table 4.

**Edge holdout.** Let $E_{\text{tt}}$ be the set of edges of $G$ whose both endpoints lie in the test-node set. We uniformly sample $20\%$ of $E_{\text{tt}}$ as held-out positive test edges.

**Negative sampling.** An equal number of test-test non-edges (unordered pairs $(u, v)$ with both endpoints in the test set and no edge in $G$) are sampled with random seed 42, shared across all methods and all seeds — so the sampled negatives are identical for every method×dataset×seed cell, and the reported ±std for deep baselines reflects encoder-training variance only, not split variance.

**Inference-time graph.** The encoder is applied to $A_{\text{lp}} = A \setminus$ (held-out positives), which preserves *all train-train edges, all train-test edges, and the remaining* $80\%$ *of test-test edges*. Validation-node edges are not separately removed.

**Edge classifier.** Logistic regression (`sklearn` defaults, `lbfgs` solver) on Hadamard products of endpoint embeddings, trained on an equal-sized sample of train-train edges (positives) and train-train non-edges (negatives). The trained classifier is then applied to the held-out positive test edges plus the sampled test-test negatives to produce AUC and AP.

**Hyperparameters.** $d$, $K$, $\lambda$ for GW-CCA are pre-set from §5 (shared with node classification); no tuning on validation edges or validation node labels is performed for either task. GW-CCA is deterministic. Deep baselines use each method's official default hyperparameters (Appendix B), also without validation-set tuning against our benchmarks.

## E Link Prediction: Average Precision

Table 8 reports the average-precision (AP) numbers corresponding to the AUC table in the main body (Table 4). GW-CCA again leads all five datasets, and per-cell agreement with AUC is close.

**Table 8:** Inductive link prediction: Average Precision (AP, %; mean ± std over 5 seeds where applicable). Deterministic methods (Raw+LR, SGC, CCA*, GW-CCA) have no seed variance. Best per column in bold.

| Method | ogbn-arxiv | Reddit | Yelp | ogbn-products | PPI |
|---|---|---|---|---|---|
| Raw features + LR | 79.6 | 76.6 | 75.3 | 84.1 | 51.2 |
| SGC [12] | 89.1 | 92.3 | 81.4 | 97.5 | 66.9 |
| GraphSAGE [22] | 95.1±0.2 | 94.8±0.8 | 87.6±0.7 | 96.4±0.2 | 81.3±0.3 |
| DGI [7] | 88.2±2.3 | 89.4±2.3 | 92.7±2.3 | 92.5±0.8 | 85.8±0.7 |
| GRACE [5] | 92.3±0.5 | 93.6±0.7 | 93.5±0.3 | 96.6±0.1 | 86.9±0.3 |
| GGD [25] | 95.2±0.2 | 92.8±1.0 | 94.2±0.0 | 98.0±0.1 | 84.8±0.6 |
| CCA-SSG [9] | 93.9±0.1 | 92.5±0.9 | 92.5±0.2 | 82.6±0.7 | 84.9±0.4 |
| BGRL [6] | 93.0±1.0 | 93.1±0.8 | 93.2±0.2 | 85.9±0.6 | 79.0±0.3 |
| GraphACL [26] | 95.2±0.2 | 90.7±0.6 | 93.6±0.1 | 96.7±0.2 | 71.8±0.7 |
| GraphMAE2 [11] | 84.4±2.3 | 87.1±0.6 | 90.8±0.4 | 96.9±0.5 | 77.6±1.9 |
| CCA* | 95.3 | 94.2 | 80.2 | 97.1 | 64.0 |
| **GW-CCA** | **97.7** | **97.2** | **95.4** | **99.6** | **89.5** |

## F Per-Line Complexity Analysis

The cost of each line of Algorithm 1 is as follows. Line 2 (computing $X_2 = \widehat{P}^K X$) performs $K$ sparse matrix–vector products at $\mathcal{O}(|E|p)$ each, totalling $\mathcal{O}(K|E|p)$. Line 3 applies $M = (P + P^2)/2$ to $X_2$ as $MX_2 = \frac{1}{2}(PX_2 + P(PX_2))$ — two rounds of sparse matrix–vector products at $\mathcal{O}(|E|p)$ each, totalling $\mathcal{O}(|E|p)$. Note that $P^2$ itself is *not* sparse in general (its nonzero count can reach $\sum_v \deg(v)^2$, potentially much larger than $|E|$ on hub-heavy graphs), so we never materialise $P^2$ or $M$; the $\mathcal{O}(|E|p)$ cost comes from left-multiplying $X_2$ by $P$ twice. Line 4 forms the two Gram matrices $C_{11} = X^\top X + \lambda I_p$ and $C_{22} = X_2^\top X_2 + \lambda I_p$ at $\mathcal{O}(np^2)$ each, and the cross-covariance $T = X^\top (MX_2)$ at $\mathcal{O}(np^2)$ (a dense $p \times n$ times $n \times p$ multiply against the already-computed $MX_2$

from line 3). Line 5 Cholesky-factorises each $p \times p$ SPD matrix at $\mathcal{O}(p^3)$. Line 6 computes the top-$d$ truncated SVD of $C_{11}^{-1/2} T C_{22}^{-1/2}$ via Lanczos-style iterations at $\mathcal{O}(p^2 d)$; the $C_{kk}^{-1/2}$ factors are applied implicitly through triangular solves against $L_k$ at $\mathcal{O}(p^2)$ per solve. Line 7 multiplies two $p \times d$ matrices at $\mathcal{O}(p^2 d)$. Summing: $\mathcal{O}((K{+}2)|E|p + np^2 + p^3)$, which we report in the main text as $\mathcal{O}(K|E|p + np^2 + p^3)$ since the two $M$-applications at line 3 add only a constant number of sparse multiplications on top of the $K$ from line 2; the bound is linear in $|E|$ and $n$. When $p$ is large, replacing the dense Cholesky at line 5 with a randomised eigensolver reduces the $\mathcal{O}(p^3)$ term.

Beyond storing the sparse graph and feature matrix, additional training memory is dominated by a constant number of $p \times p$ dense matrices ($C_{11}, C_{22}, T$, their Cholesky factors, and the top-$d$ SVD factors), an $\mathcal{O}(p^2)$ overhead. This is why GW-CCA runs in CPU RAM on graphs where GNN baselines require aggressive neighbourhood subsampling to fit on GPU.

Inductive inference (lines 8–10) costs $\mathcal{O}(K|E'|p + n'pd)$ — $K$ sparse propagations followed by two matrix multiplies of the $n' \times p$ features against the $p \times d$ projections. No training-time state beyond $\widetilde{W}, \widetilde{H}$ is needed.

# G Derivations

Two small derivations are useful for building intuition: (i) how the matrix form of the GW-CCA objective (Eq. 2) reduces to the pointwise form (Eq. 3), and (ii) how the same objective admits the SVD closed-form solution (Eq. 4).

**From the trace form to the pointwise expansion**

Write $Z_1 = X_1 \widetilde{W} \in \mathbb{R}^{n \times d}$ and $Z_2 = X_2 \widetilde{H} \in \mathbb{R}^{n \times d}$ so that the $i$-th rows are $z_1^{(i)}, z_2^{(i)} \in \mathbb{R}^d$. Then

$$\begin{aligned}\operatorname{Tr}\Big(\widetilde{W}^\top X_1^\top M X_2 \widetilde{H}\Big) &= \operatorname{Tr}\Big((X_1\widetilde{W})^\top M (X_2 \widetilde{H})\Big) = \operatorname{Tr}(Z_1^\top M Z_2) \\ &= \sum_{k=1}^{d} \sum_{i,j=1}^{n} M_{ij}\,(Z_1)_{ik}\,(Z_2)_{jk} = \sum_{i,j=1}^{n} M_{ij}\, {z_1^{(i)}}^\top z_2^{(j)},\end{aligned}$$

where the third equality expands the trace and the fourth swaps the sum orders. This is an *objective identity* only — the whitening constraints of Eq. 2 remain in force.

**From the trace form to the SVD closed form**

Set $C_{kk} = X_k^\top X_k + \lambda_k I_p$ and $T = X_1^\top M X_2$. Because $C_{kk}$ is symmetric positive definite it admits the (real, symmetric) square roots $C_{kk}^{1/2}$ and $C_{kk}^{-1/2}$. Change variables to

$$U \;=\; C_{11}^{1/2} \widetilde{W}, \qquad V \;=\; C_{22}^{1/2} \widetilde{H}.$$

The whitening constraints become $U^\top U = \widetilde{W}^\top C_{11} \widetilde{W} = I_d$ and $V^\top V = I_d$, and the objective becomes

$$\operatorname{Tr}\Big(\widetilde{W}^\top T \widetilde{H}\Big) = \operatorname{Tr}\Big(U^\top C_{11}^{-1/2} T C_{22}^{-1/2} V\Big).$$

For any matrix $B \in \mathbb{R}^{p_1 \times p_2}$, the constrained maximum $\max_{U^\top U = I_d,\, V^\top V = I_d} \operatorname{Tr}(U^\top B V)$ is attained by taking $U, V$ to be the top-$d$ left and right singular vectors of $B$ (Ky Fan's trace-maximization theorem); the maximum equals the sum of the top-$d$ singular values of $B$. Applying this to $B = C_{11}^{-1/2} T C_{22}^{-1/2}$ gives its top-$d$ truncated SVD $B = U \Sigma V^\top$, and inverting the whitening change of variables yields

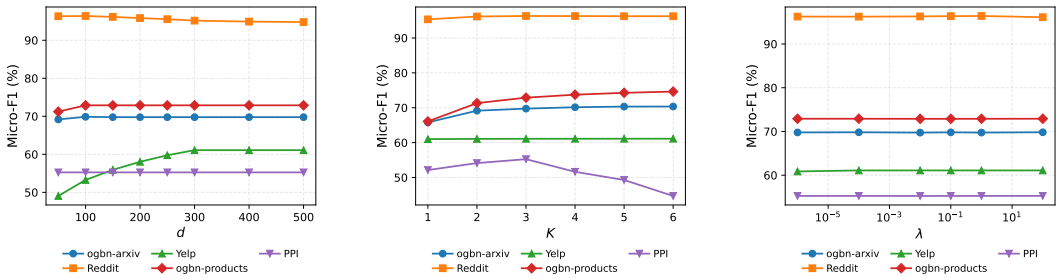
$$\widetilde{W} = C_{11}^{-1/2} U, \qquad \widetilde{H} = C_{22}^{-1/2} V,$$

which is Eq. 4. The singular values $\Sigma_{ii}$ are $M$-weighted alignment scores between the two views — analogous to canonical correlations in classical CCA, but bounded by $\|M\|_2$ rather than 1.

# H Parameter Sensitivity

We study GW-CCA's sensitivity to its three hyperparameters: dimensionality $d$, propagation depth $K$, and regularisation $\lambda = \lambda_1 = \lambda_2$. Using the fixed data splits in Table 2, Figure 2 plots Micro-F1

versus each hyperparameter while fixing the others at the defaults reported in §5. Figure 2a shows GW-CCA is insensitive to $d$ on ogbn-arxiv, ogbn-products, and PPI (the last is exactly flat because PPI's 50 raw features cap the useful embedding dimension); performance on Yelp rises with $d$ and plateaus by $d = 300$, while Reddit peaks at $d = 100$ and drifts down slowly beyond. Figure 2b shows performance saturates by $K = 3$–$4$ on Reddit, Yelp, and ogbn-arxiv; ogbn-products keeps improving monotonically through $K = 6$ ($+1.7$ pp beyond the default $K = 3$), suggesting a larger $K$ would be worth using in production; on PPI, however, $K > 3$ starts to over-smooth (Micro-F1 drops from $55.3$ at $K = 3$ to $44.7$ at $K = 6$), so $K = 3$ is a safe global default. Figure 2c shows GW-CCA is essentially insensitive to $\lambda$ across all five datasets, with variation under $\pm 0.5$ pp across six orders of magnitude ($10^{-6}$ to $10^{2}$).




**(a)** $d$ **(b)** $K$ **(c)** $\lambda$

**Figure 2:** Performance of GW-CCA versus $d$, $K$, $\lambda$ on node classification (Micro-F1, %).